AI-Enhanced BGP Autonomous Control Plane

M. Sudalai Kumar 2024503703
B. Satlas Rohit 2024503305
S. Srinivasan 2024503707
S. Ajay Kumar 2024503701

Project GitHub: https://github.com/Sudalai-kumar/AI-Enhanced-BGP

***Abstract*—The Border Gateway Protocol (BGP) enables inter-domain routing, but prefix hijacks, route leaks, and unstable announcements can disrupt reachability and redirect traffic. This paper documents an AI-enhanced BGP control-plane prototype combining routing telemetry, supervised classification, trust-based policy selection, and post-configuration verification. The repository describes a ten-AS FRRouting testbed and extracts ten features including AS-path length, path edit distance, origin-AS changes, prefix length, announcement rate, route-flap count, route age, Local Preference, valley-free violations, and neighbour diversity. Two classifiers are evaluated using a trial-based holdout artifact containing 435 samples (348 training, 87 test). Recorded Random Forest results report 85.06% accuracy and 0.4542 macro-F1, but zero F1 for the suspicious and route-leak classes. Logistic Regression reports 79.31% accuracy and 0.4301 macro-F1, with a 100% false-positive rate on a 1,000-sample normal-traffic challenge set. These results do not establish reliable multi-class detection. Repository latency values are explicitly labelled modelled dry-run estimates and are not presented as measured performance. The paper describes the architecture, available evaluation evidence, limitations, and validation requirements.**

***Index Terms*—BGP security, anomaly detection, machine learning, route hijacking, route leaks, FRRouting.**

I. Introduction

The Border Gateway Protocol exchanges reachability information between autonomous systems and selects routes according to path attributes and local policy [1]. Its distributed operation is essential to Internet connectivity, but incorrect announcements and malicious routing behaviour can redirect traffic, reduce reachability, or increase instability. Prefix hijacking, sub-prefix announcements, route leaks, and repeated route changes have different signatures and should not be treated as one anomaly class.

Research has studied BGP anomaly detection through statistical features, route embeddings, graph-based models, and operational mitigation systems [13]–[16]. These studies provide domain-specific context for feature design, anomaly detection, and operational mitigation. Their datasets and evaluation settings differ from this project's small FRRouting testbed, so their reported results are not directly comparable.

This project combines route telemetry and machine-learning classification with a policy engine intended to apply routing actions and verify resulting FRRouting state. The contributions of this paper are: (1) documentation of the telemetry-to-policy architecture; (2) reporting of available classifier results, including class-wise weaknesses; and (3) explicit separation of recorded model evaluation from modelled dry-run latency estimates.

II. Related Work

Al-Musawi *et al.* survey BGP anomaly-detection methods, features, and detection objectives [13]. Shapira and Shavitt's AP2Vec represents prefixes and AS roles to detect changes associated with possible hijacks [14]. Hoarau *et al.* study graph neural networks for identifying ASes involved in BGP anomalies [15]. ARTEMIS focuses on operational prefix-hijack detection and mitigation using monitoring data and operator-controlled actions [16]. Together, these works provide direct BGP-specific context for the design and evaluation of this prototype.

The present prototype emphasizes a combined control loop: ten hand-engineered BGP features, a classifier, a trust-to-policy mapping, and local verification. It is a testbed-specific educational prototype; its small dataset does not demonstrate generalisation to Internet-wide BGP feeds.

III. System Architecture

The repository describes ten FRRouting nodes, AS65001–AS65010, arranged as transit, regional, customer, and rogue AS roles. AS65001 is designated the core defender and AS65003 the edge defender. Scripts are provided for convergence checks, telemetry collection, training, attack injection, benchmarking, and lifecycle validation.

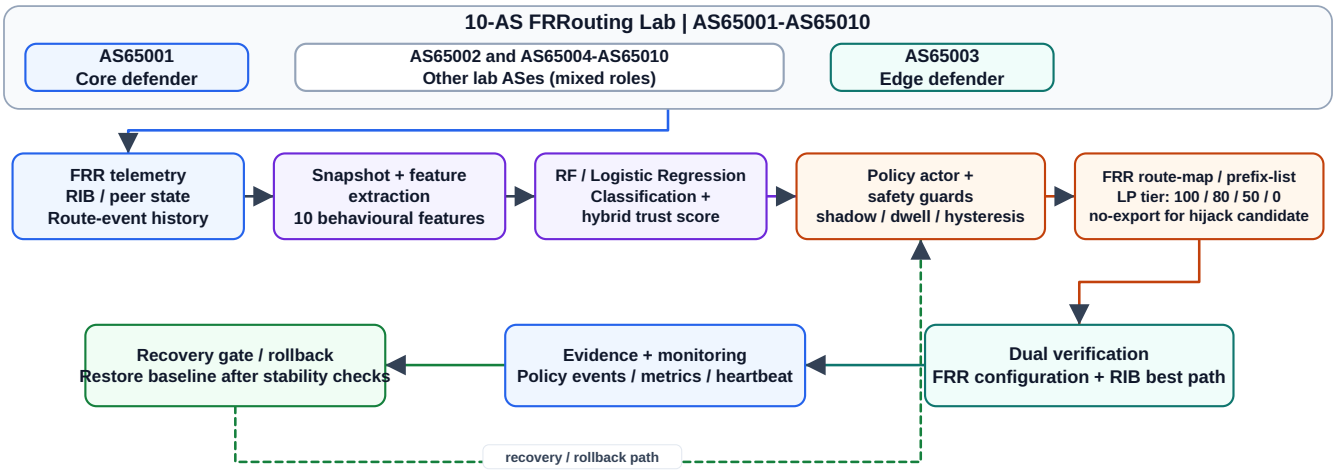


Fig. 1. Architecture diagram exported from draw.io, showing the FRRouting testbed, telemetry collection, behavioural feature extraction, classification and trust/policy logic, routing-policy enforcement, verification, and recovery. The diagram describes intended stages; it does not imply every scenario passed live validation.

A. Telemetry and Features

The extractor defines ten attributes: (1) AS-path hop count; (2) Levenshtein distance from a baseline path; (3) origin-AS change; (4) prefix mask length; (5) announcements per minute; (6) route changes within five minutes; (7) current Local Preference; (8) route age; (9) valley-free relationship violation; and (10) active announcing neighbours divided by known peers. These features represent route structure, origin changes, temporal behaviour, policy state, and AS relationships. Their value depends on representative labels and correct baselines. Route age in particular should be audited for correlations with scenario timing.

B. Classification and Policy Mapping

The implementation supports calibrated Random Forest and Logistic Regression classifiers. The policy engine documents Local Preference 100 for normal routes, 80 for suspicious routes, 50 for route-leak candidates, and 0 with a `no-export` community for hijack candidates. Hysteresis and dwell-time logic are intended to reduce policy oscillation.

Local Preference affects route selection within an AS; by itself it does not suppress repeated BGP updates. Therefore, the flapping response is described here as route deprioritisation, not protocol-level flap damping. Likewise, `no-export` restricts export according to community handling and router policy; it is not a guarantee of global route removal.

C. Verification and Recovery

The policy engine attempts to verify both the FRRouting route-map configuration and the resulting RIB best-path attributes. A lifecycle script is intended to exercise baseline operation, a sub-prefix hijack, quarantine, and rollback after removal of the rogue announcement. A successful claim requires corresponding live execution logs and observed output. Figure 2 presents the corresponding controller flow, including candidate policy selection, verification, and the recovery path.

IV. Decision Logic and Evaluation Scenarios

A. Trust-to-Policy Mapping

Let $\mathbf{x}_p(t) \in \mathbb{R}^{10}$ denote the feature vector observed for prefix p at time t. The classifier returns a class prediction c_p and calibrated class probabilities. The policy engine combines the predicted class with a trust score q_p and the currently applied Local Preference. The intended mapping is summarised in Table I. The normal state uses the default preference of 100; lower values are used to make a suspicious route less attractive than alternatives. A hijack candidate is assigned Local Preference 0 and the `no-export` community.

The code also uses class conditions and hysteresis when returning from a modified policy to the baseline. This is important because a classifier score close to a threshold can fluctuate across consecutive observations. The intended controller stages a candidate change, validates the configuration, and checks the resulting RIB

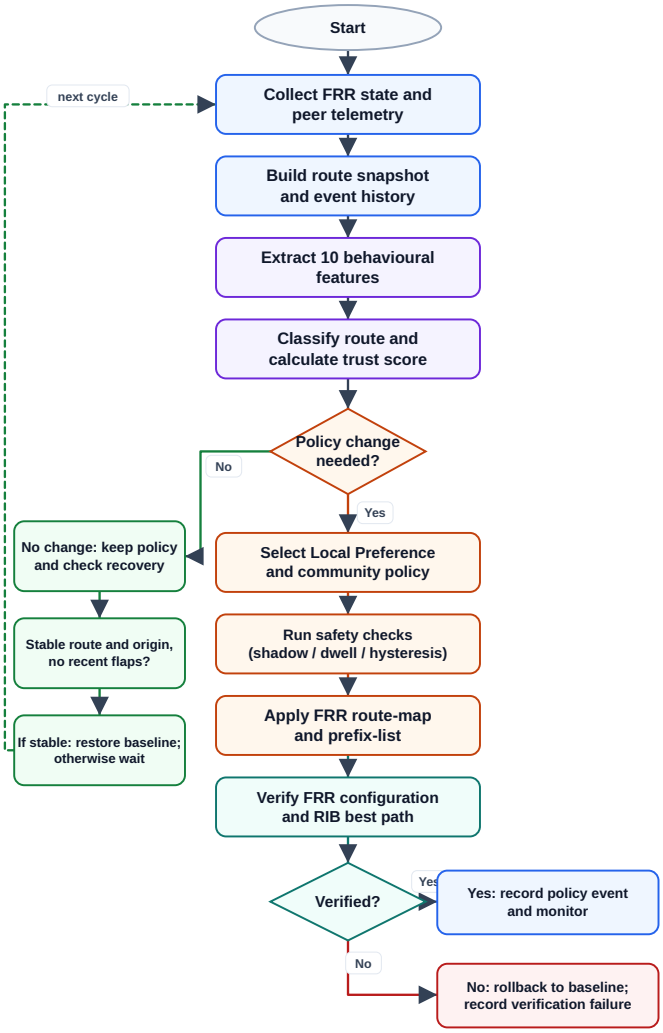


Fig. 2. Controller flow from telemetry collection through classification, policy application, verification, and recovery. The diagram shows intended logic, not proof that every scenario passed live validation.

TABLE I
DOCUMENTED TRUST/POLICY MAPPING

Trust / class condition	Policy action	Intended effect
Normal, trust ≥ 0.85	LP 100	Baseline preference
Suspicious, trust 0.55–0.80	LP 80	Soft deprioritisation
Route leak, trust 0.25–0.55	LP 50	Strong deprioritisation
Hijack, trust < 0.25 or class 3	LP 0 + `no-export`	Quarantine candidate

before treating the action as applied. The policy is still a prototype: the result depends on route-map attachment, prefix-list correctness, FRR behaviour, and the presence of alternative routes. The implementation's verification is local to the observed router and should not be described as proof of Internet-wide mitigation.

B. Attack and Replay Scenarios

The benchmark registry defines six scenarios. The first three are synthetic lab behaviours. The remaining scenarios replay a behavioural signature associated with a named historical incident within the local topology; the code explicitly states that they are not reproductions of the original Internet-scale events.

The registry is useful for a repeatable demonstration plan, but a scenario definition is not equivalent to a successful experiment. For a defensible final report, each live scenario should be accompanied by its run command, timestamp, attack-injection status, detection record, policy application result, RIB verification, cleanup/recovery result, and raw output file. Any failed

TABLE II
SCENARIO REGISTRY IN THE BENCHMARK CODE

ID	Scenario	Evidence interpretation
S1	Direct prefix hijack	Synthetic rogue-origin announcement
S2	Sub-prefix hijack (/25)	More-specific route announcement
S3	Flapping burst	Repeated route changes in a short interval
S4	YouTube 2008 signature	Topology-local origin-hijack replay
S5	Google/Rostelecom 2017 signature	Topology-local route-leak replay
S6	Cloudflare/Verizon 2019 signature	Topology-local route-leak replay

TABLE III
RECORDED CLASSIFIER EVALUATION

Metric	Random Forest	Logistic Reg.
Accuracy	85.06%	79.31%
Macro-F1	0.4542	0.4301
Weighted-F1	0.8141	0.7817
FPR (1,000 normal samples)	3.2%	100%
Suspicious-class F1	0.0000	0.0000
Route-leak-class F1	0.0000	0.0000
Hijack-class recall	83.33%	72.22%

or timed-out trial should remain in the denominator rather than being silently omitted.

C. Reproducibility Procedure

The repository's documented workflow is to start the Docker-based topology, verify that BGP sessions converge, launch the controller, and then run the relevant collection, benchmark, or lifecycle script. Model training can use generated synthetic data or the optional real-data path. These modes must be kept separate in reporting: synthetic training measures performance on the generator's distribution, whereas a testbed-derived dataset reflects only the scenarios and configurations actually sampled. A reproducibility record should therefore include the Git commit, environment versions, dataset hash, model hash, random seeds or trial IDs, command-line options, and raw output. The repository includes an environment manifest and stores model-evaluation metrics, which support provenance tracking; they do not by themselves establish that all benchmark scenarios succeeded.

D. Interpreting the Classifier Metrics

The reported Random Forest confusion matrix is

$$\begin{bmatrix} 59 & 0 & 1 & 0 \\ 3 & 0 & 0 & 0 \\ 6 & 0 & 0 & 0 \\ 2 & 0 & 1 & 15 \end{bmatrix},$$

with rows and columns ordered as Normal, Suspicious, Route Leak Candidate, and Prefix Hijack Candidate. Thus, all three Suspicious test cases and all six Route Leak Candidate cases are missed as their own classes. This explains why a weighted-F1 of 0.8141 coexists with a much lower macro-F1 of 0.4542: the dominant Normal class and the hijack class account for most of the support, while two minority classes receive zero F1. This distinction is material for a security system, because an aggregate score can conceal failure on a class the system is intended to detect. The reported challenge-set FPR also needs to be read separately from the ordinary test-set accuracy; the challenge set was designed to contain normal-like cases and exposes the Logistic Regression model's poor behaviour under that evaluation.

V. EXPERIMENTAL METHODOLOGY

A. Environment and Dataset

The repository environment manifest records Windows 11, Python 3.13.5, Docker 29.7.2, and FRRouting 10.2.1. The model-evaluation artifact identifies its source as an empirical FRRouting testbed and reports 435 samples: 348 training and 87 test samples. The split is trial-based (trials 1–4 for training and trial 5 for testing). A 1,000-sample normal-traffic challenge set is used for false-positive evaluation. Test supports are 60 Normal, 3 Suspicious, 6 Route Leak Candidate, and 18 Prefix Hijack Candidate. The small and imbalanced test set limits conclusions about minority classes.

B. Metrics and Evidence

Reported classifier metrics include accuracy, macro-F1, weighted-F1, class-wise precision/recall/F1, confusion matrices, and false-positive rate (FPR). Macro-F1 is important because it weights each class equally. The benchmark script also defines mean time to detection (MTTD) and mean time to mitigation (MTTM), measured from attack injection to the recorded detection or mitigation event.

The repository's benchmark JSON dated 26 August 2026 is explicitly marked `MODELLED (dry-run)`. Its latency values are estimates, not live measurements, and are excluded from empirical performance claims here. Classifier metrics below are reproduced from the repository's model-evaluation artifact.

VI. RESULTS AND DISCUSSION

The Random Forest has higher recorded accuracy and weighted-F1, but its macro-F1 is 0.4542. The class-wise report shows zero precision, recall, and F1 for

both Suspicious and Route Leak Candidate. It detects 15 of 18 hijack examples (83.33% recall); two are classified as Normal and one as Route Leak Candidate. Thus, it identifies many normal samples and some hijack examples but does not provide dependable four-class categorisation.

Logistic Regression reports 79.31% accuracy and 0.4301 macro-F1. Its F1 is also zero for Suspicious and Route Leak Candidate. Its FPR on the 1,000-sample normal challenge set is 1.0, meaning every challenge sample was predicted as non-normal according to the evaluation script. This is unsuitable for deployment and warrants investigation of calibration, distribution differences, and challenge-set construction.

The Random Forest feature-importance artifact assigns 0.6122 importance to route age, substantially more than any other feature. This concentration warrants scrutiny: route age may be informative, but could also encode scenario timing or collection artefacts. Ablation tests, more independent trials, and a separately collected test set are needed before claiming robust detection.

The modelled latency JSON must not be combined with classifier metrics. No empirical MTTD or MTTM is claimed here. A future live benchmark should report successful and failed trials, timeouts, mean and standard deviation, and raw output files.

VII. Operational Interpretation

A. Metric Definitions

For a multi-class classifier, accuracy is the fraction of all test instances classified correctly, while the F1 score for class k is the harmonic mean of precision and recall:

$$F1_k = 2\frac{P_k R_k}{P_k + R_k},$$

where P_k is the fraction of predictions for class k that are correct and R_k is the fraction of true class-k samples detected. Macro-F1 is the unweighted mean of the class-wise F1 scores, whereas weighted-F1 weights each class by its support. The false-positive rate used by the repository's challenge-set evaluation is the proportion of normal challenge samples predicted as any non-normal class. These metrics answer different questions: weighted-F1 reflects the observed class distribution, macro-F1 exposes weak minority-class performance, and challenge-set FPR estimates how often benign examples may trigger unnecessary policy actions under the chosen challenge distribution.

The available test support is limited, particularly for Suspicious (three instances) and Route Leak Candidate (six instances). A change of one prediction would therefore change class-wise recall substantially. The reported scores should be treated as a snapshot of the recorded experiment, not as stable estimates of expected deployment performance. Confidence intervals and repeated, independent test trials would be needed for stronger conclusions. Because the training data was collected in a controlled lab, a separate testbed configuration or public BGP trace would also help evaluate whether the classifier has learned general route behaviour rather than scenario-specific patterns.

B. Control-Plane and Data-Plane Evidence

A control-plane action and a data-plane outcome are related but distinct. Observing Local Preference and a community in a router's RIB can verify that a local route-policy change was installed and reflected in route selection. It does not establish that every AS in the path received the same policy, that all alternate paths are safe, or that end-to-end packets avoid the suspected path. A stronger evaluation should pair the RIB checks with a documented traffic probe, a defined source and destination, a pre-attack baseline, and packet-delivery measurements collected before, during, and after mitigation.

Similarly, detection latency and mitigation latency should be measured independently. MTTD is the interval between the recorded attack-injection timestamp and the first corresponding detection event. MTTM is the interval between the same start timestamp and the first confirmed mitigation event. A trial that detects an anomaly but never applies or verifies a policy must not be counted as a successful mitigation. Reports should disclose timeouts, failed injections, incomplete cleanup, and the number of successful trials. The repository benchmark code supports this separation, but its currently checked-in benchmark output is explicitly marked as a dry-run modelled result.

C. Threats to Validity

Several factors limit the strength of the findings. Internal validity may be affected by timing and state in the containerised topology, including route convergence and cleanup between trials. Construct validity is affected by whether the four labels correspond to distinct operational phenomena and whether the handcrafted feature definitions accurately capture them. The high recorded feature importance of route age makes it especially important to check for temporal shortcuts. External validity is limited by the small, topology-specific dataset and

the absence of production Internet routing data. Reproducibility also depends on using the same commit, model files, dependency versions, and experiment flags. These limitations do not make the prototype unsuitable as a course mini-project; they define the boundary between a useful demonstration and a claim of operationally reliable defence.

VIII. Limitations and Future Work

The test set has only 87 samples and few examples from two anomaly classes. Both models fail to classify Suspicious and Route Leak Candidate examples in the recorded test report, and Logistic Regression has an unacceptable challenge-set FPR. Benchmark latency values are modelled rather than empirical. Historical incident scenarios are topology-local behavioural replays, not reproductions of the original Internet-scale events. Finally, a local RIB change does not prove global containment or improved data-plane delivery.

Priority should be given to collecting more independent trials, auditing labels and class distributions, reporting per-class metrics and confidence intervals, comparing against rule-based baselines, testing feature ablations, measuring false alarms over longer normal periods, and saving logs from live attack–mitigation–recovery runs. Packet-delivery claims should only be made when the probe, traffic conditions, and baseline are documented.

IX. Conclusion

This paper documents an AI-enhanced BGP prototype combining ten behavioural features, supervised classification, Local Preference policy changes, RIB verification, and recovery logic. The recorded evaluation favours Random Forest over Logistic Regression on accuracy and weighted-F1, but both models fail to classify Suspicious and Route Leak Candidate examples in the reported test set. Logistic Regression also has a 100% false-positive rate on the normal challenge set. Because the available latency output is modelled dry-run data, it cannot support empirical mitigation-latency claims. The project is best presented as a research prototype with a clear validation gap; larger, balanced data and reproducible live measurements are needed before claiming operational reliability.

Acknowledgment

Add an acknowledgment to the course instructor or project mentor only after confirming the department's preferred wording. Do not list a mentor as co-author without their agreement and an authorship-level contribution.

References

[1] Y. Rekhter, T. Li, and S. Hares, "A Border Gateway Protocol 4 (BGP-4)," RFC 4271, Jan. 2006.

[2] R. Chandra, P. Traina, and T. Li, "BGP Communities Attribute," RFC 1997, Aug. 1996.

[3] R. Mohapatra, J. Scudder, D. Ward, R. Bush, and R. Austein, "BGP Prefix Origin Validation," RFC 6811, Jan. 2013.

[4] D. Heitz, C. S. Schmidt, and J. T. Schiller, "Route Leak Prevention and Detection Using BGP Communities," RFC 9234, May 2022.

[5] L. Gao and J. Rexford, "Stable Internet routing without global coordination," *IEEE/ACM Transactions on Networking*, vol. 9, no. 6, pp. 681–692, Dec. 2001.

[6] X. Fan and J. Heidemann, "BGP routing dynamics during the YouTube incident," in *Proc. ACM/USENIX IMC*, 2008.

[7] R. Bush and R. Austein, "The RPKI and route origin validation," IETF RFC 6811/6907 family of operational guidance, 2013.

[8] P. Marantis, K. Ramantas, and C. V. Verikoukis, "Anomaly Detection With Quantified Explanations for Multivariate Time Series Networking Data," in *Proc. IEEE GLOBECOM*, 2024, doi: 10.1109/GLOBECOM52923.2024.10901585.

[9] M.-M. Nikula, L. M. Correia, A. Grilo, P. Mähönen, L. Santo, and R. Dinis, "Machine Learning-Based Anomaly Detection and Root Cause Analysis in Mobile Networks," in *Proc. IEEE WCNC*, 2025, doi: 10.1109/WCNC61545.2025.10978282.

[10] A. Karhana, I. Saini, and A. Jaekel, "Machine Learning Approach to False Alert Attack Detection in Vehicular Adhoc Networks," in *Proc. IEEE CCNC*, 2025, pp. 1–7, doi: 10.1109/CCNC54725.2025.10975893.

[11] D. Wu, Z. Peng, M. Chen, and Y. Liu, "Transforming Network Intrusion Detection Using Large Language Models," in *Proc. IEEE CCNC*, 2025, pp. 1–9, doi: 10.1109/CCNC54725.2025.10976105.

[12] Y. Liu, Z. Zhang, X. Liu, H. Zhou, and L. Zhao, "PH-BGP: A Proactive Hierarchical Border Gateway Protocol Routing Scheme and System in Ultra-Dense LEO Satellite Network," in *Proc. IEEE ICC*, 2025, pp. 5902–5907, doi: 10.1109/ICC52391.2025.11161107.

[13] A. Al-Musawi, M. Branch, and G. Armitage, "BGP Anomaly Detection Techniques: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 1, pp. 377–396, 2017.

[14] Y. Shapira and Y. Shavitt, "AP2Vec: An Unsupervised Approach for BGP Hijacking Detection," *IEEE Transactions on Network and Service Management*, 2022, doi: 10.1109/TNSM.2022.3166450.

[15] T. Hoarau *et al.*, "BGNN: Detection of BGP Anomalies Using Graph Neural Networks," in *Proc. IEEE International Symposium on Computers and Communications (ISCC)*, 2022, doi: 10.1109/ISCC55528.2022.9912989.

[16] P. Sermpezis *et al.*, "ARTEMIS: Neutralizing BGP Hijacking Within a Minute," *IEEE/ACM Transactions on Networking*, vol. 26, no. 6, pp. 2471–2486, 2018, doi: 10.1109/TNET.2018.2869798.